

USERS AND GROUPS

Linux is a multi-user system. Users and groups help you control who can access the system and what they can do.

KEY CONCEPTS



USER

- An account that allows a person to access the system.
- Has a unique User ID (UID).
- Owns files and processes.



GROUP

- A collection of users.
- Has a unique Group ID (GID).
- Simplifies permission management.



Every user belongs to at least one group (their primary group) and can belong to multiple additional groups.



USER AND GROUP FILES

<code>/etc/passwd</code>	Stores user account information
<code>/etc/shadow</code>	Stores encrypted passwords
<code>/etc/group</code>	Stores group information
<code>/etc/gshadow</code>	Stores group passwords



These files are critical to the system. Edit them only with proper tools.

COMMON COMMANDS



USER MANAGEMENT

COMMAND	DESCRIPTION	EXAMPLE
<code>useradd username</code>	Create a new user	<code>sudo useradd john</code>
<code>useradd -m username</code>	Create user with home directory	<code>sudo useradd -m jane</code>
<code>passwd username</code>	Set or change user password	<code>sudo passwd john</code>
<code>usermod -aG group user</code>	Add user to a group	<code>sudo usermod -aG sudo john</code>
<code>usermod -l newname oldname</code>	Rename a user	<code>sudo usermod -l john_new john</code>
<code>usermod -d /new/home user</code>	Change user home directory	<code>sudo usermod -d /home/john john</code>
<code>userdel username</code>	Delete a user	<code>sudo userdel john</code>
<code>userdel -r username</code>	Delete user and home directory	<code>sudo userdel -r john</code>
<code>id username</code>	Show user ID and groups	<code>id john</code>
<code>who</code>	Chow logged in users	<code>who</code>
<code>w</code>	Show who is logged in and what they are doing	<code>w</code>



GROUP MANAGEMENT

COMMAND	DESCRIPTION	EXAMPLE
<code>groupadd groupname</code>	Create a new group	<code>sudo groupadd devteam</code>
<code>groupmod -n newname oldname</code>	Rename a group	<code>sudo groupmod -n dev devteam</code>
<code>groupdel groupname</code>	Delete a group	<code>sudo groupdel devteam</code>
<code>gpasswd -a user group</code>	Add user to a group	<code>sudo gpasswd -a john dev</code>
<code>gpasswd -d user group</code>	Remove user from a group	<code>sudo gpasswd -d john dev</code>
<code>groups username</code>	Show groups a user belongs to	<code>groups john</code>
<code>getent group groupname</code>	Display group information	<code>getent group dev</code>



CHECK USERS AND GROUPS

<code>cat /etc/passwd</code>	View all users
<code>cat /etc/group</code>	View all groups
<code>getent passwd</code>	List all users
<code>getent group</code>	List all groups



BEST PRACTICES

- ✓ Use strong passwords and change regularly.
- ✓ Follow the principle of least privilege.
- ✓ Add users to groups instead of giving individual permissions.
- ✓ Review user and group access regularly.
- ✓ Disable or remove unused accounts.



QUICK TIP

Use groups to manage permissions easily. For example, add all developers to a "dev" group and give the group access to project files.



FILE OWNERSHIP AND PERMISSIONS (QUICK RECAP)

Every file has an owner and a group.

Change owner

```
chown user:group filename
```

Change group

```
chgrp groupname filename
```

Change permissions

```
chmod 755 filename
```

> Check ownership

```
ls -l filename
```



USEFUL COMMANDS

<code>su - username</code>	Switch to another user
<code>sudo command</code>	Run command as another user
<code>whoami</code>	Show current logged in user
<code>last</code>	Show last logged in users
<code>newgrp groupname</code>	Switch to a new primary group



LINUX PERMISSIONS AND OWNERSHIP

Linux uses permissions and ownership to protect files and directories and control who can read, write or execute them.

3. PERMISSION TYPES

Symbol	Meaning	Value	Description
r	Read	4	View file contents, list directory contents
w	Write	2	Modify file contents, create/delete files in directory
x	Execute	1	Run file as a program, enter directory
-	None	0	No permission

4. NUMERIC PERMISSIONS

Permissions can be represented as numbers (r=4, w=2, x=1). Add the values to get the final number.
Example:

Permission	Owner	Group	Others	Numeric
rwX r-x r--	4+2+1 = 7	4+0+1 = 5	4+0+0 = 4	754
rw- r-- ---	4+2+0 = 6	4+0+0 = 4	0+0+0 = 0	640
rwX rwX r-x	7	7	5	775

Special Permission Bits

Setuid (4xxx)	Run as the file owner
Setgid (2xxx)	Run as the file group
Sticky bit (1xxx)	Restrict deletion to file owner in a directory

QUICK REFERENCE

r = 4 (read)	rwX = 7
w = 2 (write)	rw- = 6
x = 1 (execute)	r-- = 4
	--- = 0

1. KEY CONCEPTS



OWNERSHIP

Every file and directory has an owner and a group.

- **Owner:** The user who owns the file.
- **Group:** A collection of users.



PERMISSIONS

Define what the owner, group and others can do with a file or directory.

- r (read) 4
- w (write) 2
- x (execute) 1



2. THE PERMISSION MODEL

Each file/directory has 3 permission sets:



Example: -rwxr-xr--

5. COMMON COMMANDS

OWNERSHIP COMMANDS

Command	Description
ls -l	List files with permissions and ownership
whoami	Show current user
id	Show user and group IDs
chown user file	Change owner of a file
chown user:group file	Change owner and group
chgrp group file	Change group of a file
chown -R user:group dir/	Change owner/group recursively

PERMISSION COMMANDS

Command	Description
chmod mode file	Change permissions (numeric mode)
chmod u+r file	Add read permission to owner
chmod g-w file	Remove write permission from group
chmod o+x file	Add execute permission to others
chmod -R 755 dir/	Change permissions recursively
umask	Show default permission mask
umask 027	Set default permission mask

6. EXAMPLES

ls -l file.txt	View permission and ownership
chmod 644 file.txt	rw-r--r-- (owner rw, group r, others r)
chmod 755 script.sh	rwxr-xr-x (add execute for all)
chmod u+x script.sh	Add execute for owner only
chmod go-w file.txt	Remove write from group and others
chown john file.txt	Change owner to john
chown john:dev file.txt	Change owner to john and group to dev
chgrp dev file.txt	Change group to dev

8. BEST PRACTICES

- ✓ Follow the principle of least privilege.
- ✓ Do not give unnecessary write or execute permissions.
- ✓ Use groups to manage access for multiple users.
- ✓ Regularly audit permissions and ownership.
- ✓ Be careful with recursive changes (chown/chmod -R).

7. FILE TYPE INDICATORS (ls -l)

- Regular file
- d Directory
- l Symbolic link
- c Character device
- b Block device
- p Named pipe
- s Socket

Example: ls -l

-rwxr-xr--	1	john	dev	1024	May 20 10:00	script.sh
Permissions	Links	Owner	Group	Size	Date	Name

DEFAULT PERMISSIONS FOR NEW FILES & DIRECTORIES

Files (umask 022)	Directories (umask 022)
Default: 644 (rw-r--r--)	Default: 755 (rwxr-xr-x)



TIP

Permissions control "what" users can do. Ownership controls "who" has control. Use them together to secure your system!



ROOT, SUDO AND PRIVILEGE MANAGEMENT

Linux is a multi-user system. Privilege management ensures that users have the right level of access to perform tasks securely and safely.



KEY CONCEPTS



ROOT

The root user (UID 0) has unrestricted access to the entire system.

- Can do anything
- Use with extreme care
- Work as root only when necessary



SUDO

sudo (superuser do) allows permitted users to run commands as another user, by default as root.

- Provides controlled access
- All actions are logged
- Reduces risk



PRIVILEGE MANAGEMENT

The practice of controlling who can do what on the system.

- Least privilege principle
- Role-based access
- Audit and monitor



Use the minimum privilege necessary to complete a task. Do not operate as root for everyday work.

ROOT vs SUDO vs NORMAL USER

Feature	Root User	Sudo User	Normal User
User ID	0	> 0 (in sudoers)	> 0 (not in sudoers)
Access Level	Unrestricted	Limited (as per sudoers rules)	Limited
How to Use	Login as root or su -	sudo <command>	Run commands without elevation
Risk	Very High	Medium (controlled)	Low
Logging	Limited by default	Logged in /var/log/auth.log	Logged (user activities)
Best Use	System admin maintenance	Daily admin tasks with control	Regular work and applications

COMMON COMMANDS



WORKING WITH ROOT

```
su -           # Switch to root user
su - root     # Login as root
whoami        # Show current user
id            # Show user and group IDs
exit          # Exit to previous user
```



USING SUDO

```
sudo command  # Run command as root
sudo -i       # Start root shell
sudo -s       # Start root shell (sh)
sudo -u user command # Run as another user
sudo !!       # Repeat last command with sudo
sudo -v       # Refresh sudo timestamp
```

Note:
Avoid staying logged in as root for long periods.

After authentication, sudo allows multiple commands for a short time (default 5 minutes).



SUDO UTILITIES

```
sudo -l       # List allowed commands
sudo -ll      # Detailed list (with hosts)
sudo visudo   # Edit sudoers file safely
sudo passwd root # Set root password
sudo passwd username # Change user password
```

Use visudo to edit sudoers. Never edit /etc/sudoers directly.

THE SUDOERS FILE

The file /etc/sudoers (and files in /etc/sudoers.d/) defines who can run which commands with sudo.

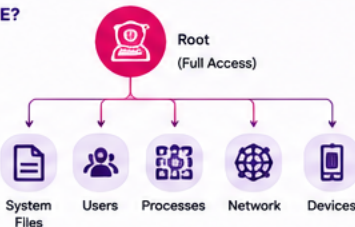
```
# User privilege specification
username ALL=(ALL:ALL) ALL # User can run all commands
username ALL=(root) /usr/bin/systemctl restart nginx # Specific command
%groupname ALL=(ALL) NOPASSWD: ALL # Group can run all without password
Defaults env_reset # Reset environment
Defaults secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

Syntax: user host=(run_as_user:run_as_group) command(s)



WHY NOT USE ROOT ALL THE TIME?

- ✓ Mistakes can cause system-wide damage.
- ✓ Malware or stolen sessions have full control.
- ✓ Harder to audit and track actions.
- ✓ Violates the principle of least privilege.



BEST PRACTICES

- ✓ Use sudo for administrative tasks.
- ✓ Create dedicated users and grant specific access.
- ✓ Do not share root passwords.
- ✓ Regularly review sudoers permissions.
- ✓ Enable logging and monitor admin activities.
- ✓ Use strong passwords and MFA where possible.



SECURITY TIPS

- ✓ Keep your system and sudo packages updated.
- ✓ Use long timeout for sudo only when necessary.
- ✓ Limit NOPASSWD usage to specific commands.
- ✓ Monitor /var/log/auth.log for sudo activity.
- ✓ Remove unused accounts and sudo access.



QUICK TIP

Give users just enough privilege to do their job—nothing more.

More security, more accountability!



PACKAGE MANAGEMENT



Package management tools help you install, upgrade, configure and remove software on Linux systems efficiently and securely.

KEY CONCEPTS



Package

A software bundle that contains program files, configuration, scripts and metadata.



Repository

A collection of packages stored on remote servers (from where packages are downloaded).



Dependency

Packages often rely on other packages to work. The manager resolves and installs them automatically.



Benefits

Consistency, security updates, easy upgrades, and dependency handling.

COMMON PACKAGE MANAGERS

Distribution Family	Examples	Package Manager	Configuration / Repo Path
Debian / Ubuntu	Ubuntu, Debian, Kali, Linux Mint, Pop!_OS	apt / apt-get (Advanced Package Tool)	/etc/apt/sources.list /etc/apt/sources.list.d/
RHEL / CentOS / Fedora	RHEL, CentOS Stream, Fedora, Rocky Linux, AlmaLinux	dnf (new) / yum (legacy)	/etc/yum.repos.d/
SUSE	openSUSE, SUSE Linux Enterprise	zypper	/etc/zypp/repos.d/
Arch Linux	Arch Linux, Manjaro	pacman	/etc/pacman.conf

COMMAND REFERENCE

DEBIAN / UBUNTU (apt / apt-get)

Task	Command	Description
Update package index	<code>sudo apt update</code>	Download the latest package list
Upgrade packages	<code>sudo apt upgrade</code>	Upgrade installed packages
Upgrade all packages	<code>sudo apt full-upgrade</code>	Upgrade all including new dependencies
Install a package	<code>sudo apt install <package></code>	Install a package
Remove a package	<code>sudo apt remove <package></code>	Remove a package (keeps config files)
Purge a package	<code>sudo apt purge <package></code>	Remove package and config files
Search for a package	<code>apt search <keyword></code>	Search in available packages
Show package info	<code>apt show <package></code>	Display package information
List installed packages	<code>dpkg -l</code>	List all installed packages
Clean cache	<code>sudo apt clean</code>	Remove downloaded package files
Autoremove unused deps	<code>sudo apt autoremove</code>	Remove no longer required packages

RHEL / CENTOS / FEDORA (dnf / yum)

Task	Command (dnf)	Description
Update package index	<code>sudo dnf check-update</code>	Check for available updates
Upgrade packages	<code>sudo dnf upgrade</code>	Upgrade installed packages
Upgrade all packages	<code>sudo dnf upgrade --refresh</code>	Upgrade all packages
Install a package	<code>sudo dnf install <package></code>	Install a package
Remove a package	<code>sudo dnf remove <package></code>	Remove a package
Search for a package	<code>dnf search <keyword></code>	Search in available packages
Show package info	<code>dnf info <package></code>	Display package information
List installed packages	<code>dnf list installed</code>	List installed packages
Clean cache	<code>sudo dnf clean all</code>	Clean all cached data
Remove obsolete deps	<code>sudo dnf autoremove</code>	Remove unused dependencies

SUSE (zypper)

Task	Command	Description
Refresh repositories	<code>sudo zypper refresh</code>	Refresh repository metadata
Upgrade packages	<code>sudo zypper update</code>	Update installed packages
Install a package	<code>sudo zypper install <package></code>	Install a package
Remove a package	<code>sudo zypper remove <package></code>	Remove a package
Search for a package	<code>zypper search <keyword></code>	Search in available packages
Show package info	<code>zypper info <package></code>	Display package information
List installed packages	<code>zypper list-installed</code>	List installed packages
Clean cache	<code>sudo zypper clean</code>	Clean downloaded packages



BEST PRACTICES

- Always update package index before installing or upgrading.
- Keep your system updated regularly.
- Use official repositories when possible.
- Review package details before installing.
- Remove unused packages and clean cache to save space.
- Backup important data before major upgrades.



USEFUL EXAMPLES

```
sudo apt update && sudo apt upgrade -y # Update and upgrade (Debian/Ubuntu)
sudo dnf install httpd vim git          # Install multiple packages (dnf)
sudo apt autoremove && sudo apt clean    # Clean and remove unused (apt)
sudo dnf upgrade --refresh              # Refresh and upgrade all (dnf)
```



CHECK PACKAGE VERSION

```
apt show <package> | grep Version # Debian/Ubuntu
dnf info <package> | grep Version  # RHEL/Fedora
zypper info <package> | grep Version # SUSE
```



QUICK TIP

If a package is not found, try updating the repository index and searching again.

```
sudo apt update && apt search <package>
sudo dnf check-update && dnf search <package>
sudo zypper refresh && zypper search <package>
```



PROCESSES AND JOB MANAGEMENT

Linux is a multi-tasking system. It runs many processes in the background. These tools help you view, manage and control them effectively.



KEY CONCEPTS



PROCESS

A running instance of a program. Every process has a Process ID (PID).



JOB

A process or group of processes managed by the shell.



FOREGROUND PROCESS

Runs in the terminal and takes input until it finishes.



BACKGROUND PROCESS

Runs independently in the background and does not use the terminal.



Use processes for system tasks and jobs for shell command control.

VIEWING PROCESSES

COMMAND	DESCRIPTION	EXAMPLE
<code>ps</code>	Show snapshot of current processes	<code>ps</code>
<code>ps aux</code>	Show all processes (detailed)	<code>ps aux</code>
<code>top</code>	Interactive process viewer	<code>top</code>
<code>htop</code>	Enhanced interactive process viewer (may need installation)	<code>htop</code>
<code>pgrep <name></code>	Search for process by name	<code>pgrep nginx</code>
<code>pidof <name></code>	Show PID of process	<code>pidof sshd</code>

ps aux OUTPUT EXPLAINED	
COLUMN	MEANING
USER	User who started the process
PID	Process ID
%CPU	CPU usage
%MEM	Memory usage
VSZ	Virtual memory size
RSS	Resident memory size
TTY	Terminal (or ? for none)
STAT	Process status
START	Start time
TIME	CPU time used
COMMAND	Command that started the process

PROCESS MANAGEMENT COMMANDS

COMMAND	DESCRIPTION	EXAMPLE
<code>kill <PID></code>	Terminate a process (default SIGTERM)	<code>kill 1234</code>
<code>kill -9 <PID></code>	Force kill a process (SIGKILL)	<code>kill -9 1234</code>
<code>pkill <name></code>	Kill processes by name	<code>pkill nginx</code>
<code>killall <name></code>	Kill all processes with the name	<code>killall apache2</code>
<code>nice -n <num> <cmd></code>	Start a process with adjusted priority (lower value = higher priority)	<code>nice -n 10 tar -czf backup.tgz /data</code>
<code>renice <num> -p <PID></code>	Change priority of a running process	<code>renice -n 5 -p 1234</code>

Use `kill -9` only when necessary. It cannot be caught or ignored by the process.

JOB MANAGEMENT (SHELL JOBS)

COMMAND	DESCRIPTION	EXAMPLE
<code>command &</code>	Run command in background	<code>sleep 100 &</code>
<code>jobs</code>	List background and stopped jobs	<code>jobs</code>
<code>fg %jobid</code>	Bring job to foreground	<code>fg %1</code>
<code>bg %jobid</code>	Resume job in background	<code>bg %1</code>
<code>Ctrl + Z</code>	Suspend foreground job	(Press <code>Ctrl + Z</code>)
<code>kill %jobid</code>	Kill a job	<code>kill %1</code>
<code>disown %jobid</code>	Remove job from shell job table	<code>disown %1</code>

Job ID is usually shown as [number] or %number (e.g., %1, %2).

JOB CONTROL EXAMPLES

- Run a long task in background
`$ tar -czf backup.tgz /data &` # Job started in background
- Check running jobs
`$ jobs` # See job list
- Pause the job
`$ Ctrl + Z` # Job stopped
- Resume in background
`$ bg %1` # Job continues in background
- Bring to foreground
`$ fg %1` # Job is now in foreground

PROCESS STATES (STAT COLUMN)

R	Running or runnable
S	Sleeping (interruptible)
D	Uninterruptible sleep (usually I/O)
T	Stopped (by job control signal)
Z	Zombie (terminated but not reaped)
<	High priority
N	Low priority
+	In foreground process group

EXAMPLE

```
$ ps aux | head -n 1
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
$ ps aux | grep nginx
root      1234  0.0  0.1 22500 4200 ?        Ss   10:00   0:00 nginx: master process
www-data  1235  0.1  0.2 23100 5200 ?        S    10:00   0:01 nginx: worker process
```

Zombie processes can be identified by "Z". They should be eliminated by their parent process.

USEFUL COMMANDS QUICK LIST

<code>uptime</code>	Show system uptime and load average
<code>w</code>	Show who is logged in and what they are doing
<code>whoami</code>	Show current user
<code>time command</code>	Measure how long a command takes
<code>free -h</code>	Show memory usage
<code>df -h</code>	Show disk usage
<code>vmstat 1 5</code>	System performance report (every 1 sec, 5 times)

BEST PRACTICES

- Always check before killing a process.
- Use `nice` and `renice` to manage priority instead of killing.
- Run long tasks in background.
- Monitor system resources regularly.
- Avoid leaving zombie processes.



QUICK TIP

Use `htop` for a better, user-friendly view of processes.

Install: `sudo apt install htop` (Debian/Ubuntu)
`sudo dnf install htop` (RHEL/CentOS/Fedora)

LINUX SERVICES AND SYSTEMD



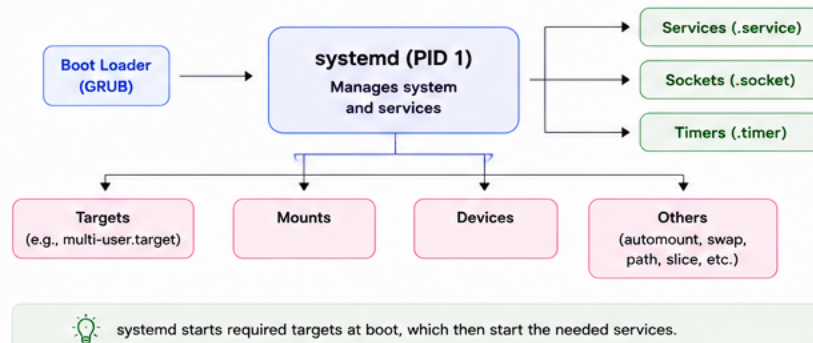
systemd is the init system and service manager for most modern Linux distributions. It starts services at boot, supervises them, and provides a consistent way to manage system processes and resources.

KEY CONCEPTS

- SERVICE** A long-running process that performs a specific task (e.g., web server, database).
- SYSTEMD** The system and service manager. PID 1 (init) on systemd-based systems.
- UNIT** systemd manages "units". The most common unit type for services is .service files.
- TARGET** Groups of units used to define boot states (e.g., multi-user.target, graphical.target).

i Most commands require root privileges. Use **sudo**.

SYSTEMD ARCHITECTURE



SERVICE MANAGEMENT COMMANDS

COMMAND	DESCRIPTION	EXAMPLE
<code>systemctl start <service></code>	Start a service	<code>sudo systemctl start nginx</code>
<code>systemctl stop <service></code>	Stop a service	<code>sudo systemctl stop nginx</code>
<code>systemctl restart <service></code>	Restart a service	<code>sudo systemctl restart nginx</code>
<code>systemctl reload <service></code>	Reload service configuration	<code>sudo systemctl reload nginx</code>
<code>systemctl enable <service></code>	Enable service to start at boot	<code>sudo systemctl enable nginx</code>
<code>systemctl disable <service></code>	Disable service from starting at boot	<code>sudo systemctl disable nginx</code>
<code>systemctl status <service></code>	Show service status	<code>systemctl status nginx</code>
<code>systemctl is-enabled <service></code>	Check if service is enabled	<code>systemctl is-enabled nginx</code>

i Enabling a service does not start it now; it only starts on boot.

VIEW SERVICES AND UNITS

COMMAND	DESCRIPTION	EXAMPLE
<code>systemctl list-units --type=service</code>	List all loaded services	<code>systemctl list-units --type=service</code>
<code>systemctl list-units --type=service --state=running</code>	List running services	<code>systemctl list-units --type=service --state=running</code>
<code>systemctl --failed</code>	Show failed units	<code>systemctl --failed</code>
<code>systemctl list-dependencies <service></code>	Show dependencies of a service	<code>systemctl list-dependencies nginx.service</code>
<code>systemctl list-unit-files</code>	List all unit files and their status	<code>systemctl list-unit-files</code>
<code>systemctl cat <service></code>	Show unit file content	<code>systemctl cat nginx.service</code>

COMMON TARGETS

TARGET	DESCRIPTION
<code>default.target</code>	Default target (usually points to graphical.target or multi-user.target)
<code>multi-user.target</code>	Multi-user text mode (no GUI)
<code>graphical.target</code>	Graphical mode (GUI login)
<code>rescue.target</code>	Rescue mode
<code>reboot.target</code>	Reboot the system
<code>poweroff.target</code>	Power off the system
<code>emergency.target</code>	Emergency shell

SERVICE STATUS EXPLAINED

STATUS	MEANING
● active (running)	Service is running normally
● active (exited)	Service ran and exited successfully (e.g., oneshot)
● inactive (dead)	Service is stopped
● failed	Service failed to start
● activating	Service is starting
● deactivating	Service is stopping

Example: `systemctl status nginx`

```
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2024-05-14 10:15:23 MAT; 2h 10min ago
     Docs: man:nginx(8)
    Main PID: 1234 (nginx)
      Tasks: 3 (Limit: 4634)
    Memory: 5.2M
       CPU: 120ms
    CGroup: /system.slice/nginx.service
            └─1234 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
            └─1235 nginx: worker process
            └─1236 nginx: worker process
```

✓ Check logs with: `journalctl -u nginx -xe`

UNIT FILE LOCATIONS

PATH	PURPOSE
<code>/usr/lib/systemd/system/</code>	Vendor (packaged) unit files
<code>/etc/systemd/system/</code>	System-wide custom unit files (override or new)
<code>/etc/systemd/system/*.d/</code>	Drop-in override configurations
<code>/run/systemd/system/</code>	Runtime unit files (generated)
<code>~/.config/systemd/user/</code>	User-level unit files

⚡ After changing or creating unit files, run:
`sudo systemctl daemon-reload`

BEST PRACTICES

- ✓ Enable only necessary services.
- ✓ Review service status and logs regularly.
- ✓ Use specific targets for different boot modes.
- ✓ Avoid disabling essential system services.
- ✓ Create custom services using unit files.
- ✓ Keep your system updated.



QUICK TIP

To edit a systemd service file override:

`sudo systemctl edit <service>`

This creates a drop-in file in `/etc/systemd/system/<service>.d/`

LINUX NETWORKING FOR DEVOPS

Linux provides powerful networking tools and configuration options that DevOps engineers use daily to build, secure and operate reliable systems and applications.



KEY CONCEPTS



NETWORK

A collection of devices connected to share data and resources.



INTERFACE

A virtual or physical network connection (e.g., eth0, ens33).



IP ADDRESS

Logical address that identifies a device in a network (IPv4 or IPv6).



ROUTING

The process of selecting the best path for packets to reach their destination.



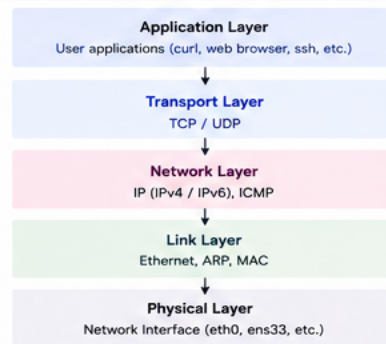
DNS

Translates domain names into IP addresses.



Understanding these concepts is essential for automation, troubleshooting and system reliability.

NETWORKING STACK IN LINUX



Linux implements the full TCP/IP stack and provides tools to manage each layer.

COMMON NETWORK INTERFACES

COMMAND	DESCRIPTION	EXAMPLE
<code>ip link</code>	Show / manage network interfaces	<code>ip link</code>
<code>ip addr</code>	Show / manage IP addresses	<code>ip addr show</code>
<code>ip route</code>	Show / manage routing table	<code>ip route show</code>
<code>ss</code>	Display socket statistics	<code>ss -tulnp</code>
<code>ping</code>	Test reachability to a host	<code>ping 8.8.8.8</code>
<code>traceroute</code>	Trace packet route to a host	<code>traceroute google.com</code>
<code>dig / host / nslookup</code>	DNS lookup tools	<code>dig google.com</code>
<code>arp</code>	Show / manipulate ARP cache	<code>arp -a</code>
<code>ethtool</code>	Show / manage NIC settings	<code>ethtool eth0</code>
<code>nmcli</code>	NetworkManager command line	<code>nmcli device status</code>



CONFIGURING IP ADDRESS (TEMPORARY)

IPv4

```
# Add IP address
sudo ip addr add 192.168.1.10/24 dev eth0

# Bring interface up
sudo ip link set eth0 up

# View IP address
ip addr show eth0
```

IPv6

```
# Add IPv6 address
sudo ip -6 addr add 2001:db8::10/64 dev eth0

# Bring interface up
sudo ip link set eth0 up

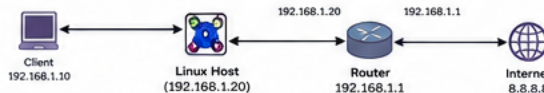
# View IPv6 address
ip -6 addr show eth0
```

For permanent configuration, use files under `/etc/netplan/`, `/etc/network/interfaces` or NetworkManager (`nmcli`).



ROUTING BASICS

COMMAND	DESCRIPTION / EXAMPLE
<code>ip route show</code>	Show routing table
<code>ip route add <network> via <gateway></code>	Add a route <code>ip route add 10.10.0.0/16 via 192.168.1.1</code>
<code>ip route del <network></code>	Delete a route
<code>ip route add default via <gateway></code>	Set default gateway <code>ip route add default via 192.168.1.1</code>



Traffic to unknown networks is sent to the default gateway.



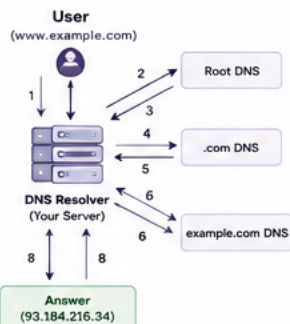
CHECK NETWORK CONNECTIVITY

TOOL / COMMAND	PURPOSE / EXAMPLE
<code>ping <host></code>	Check if host is reachable <code>ping 8.8.8.8</code>
<code>ping -c 4 <host></code>	Send 4 packets and stop
<code>traceroute <host></code>	Trace path to host
<code>mtr <host></code>	Combined traceroute + ping (install mtr)
<code>curl <url></code>	Test HTTP/HTTPS connectivity <code>curl -I https://google.com</code>
<code>nc -zv <host> <port></code>	Check if port is open <code>nc -zv google.com 80</code>
<code>ss -tulnp</code>	List listening ports and services



DNS RESOLUTION

COMMAND	DESCRIPTION / EXAMPLE
<code>cat /etc/resolv.conf</code>	View DNS configuration
<code>dig <domain></code>	Detailed DNS lookup <code>dig google.com</code>
<code>host <domain></code>	Simple DNS lookup <code>host google.com</code>
<code>nslookup <domain></code>	Interactive DNS lookup <code>nslookup google.com</code>



COMMON NETWORK PORTS

PORT	PROTOCOL	SERVICE	DESCRIPTION
22	TCP	SSH	Secure remote access
80	TCP	HTTP	Web traffic (unencrypted)
443	TCP	HTTPS	Web traffic (encrypted)
53	TCP/UDP	DNS	Domain Name System
25	TCP	SMTP	Email sending
110	TCP	POP3	Email retrieval
3306	TCP	MySQL	MySQL database
5432	TCP	PostgreSQL	PostgreSQL database

Use `ss -tulnp` to see which services are listening on ports.



FIREWALL BASICS (FIREWALLD / IPTABLES)

firewalld (RHEL/CentOS/Fedora)

```
sudo firewall-cmd --list-all
sudo firewall-cmd --add-service=ssh --permanent
sudo firewall-cmd --reload
```

Iptables (Legacy)

```
sudo iptables -L -n -v
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables-save
```

Always restrict access to only necessary ports and trusted networks.



BEST PRACTICES

- Use static IPs for servers and critical systems.
- Keep `/etc/hosts`, DNS and routing configured correctly.
- Open only required ports and close unused services.
- Monitor network connections and bandwidth usage.
- Automate network configuration with tools like Ansible.



QUICK TIP

`ip a` is a handy shortcut for `ip addr show`.

`ip a`



LINUX PROCESS MANAGEMENT



Linux is a multi-tasking system. Process management tools help you monitor, control and optimize system processes for better performance and reliability.

KEY CONCEPTS



PROCESS

A running instance of a program. Every process has a PID.



JOB

A process or group of processes managed by the shell.



FOREGROUND PROCESS

Runs in the terminal and takes input until it finishes.



BACKGROUND PROCESS

Runs independently in the background and does not use the terminal.



Use processes for system tasks and jobs for shell command control.

VIEWING PROCESSES

COMMAND	DESCRIPTION	EXAMPLE
ps	Show snapshot of current processes	<code>ps</code>
ps aux	Show all processes (detailed)	<code>ps aux</code>
top	Interactive process viewer	<code>top</code>
htop	Enhanced interactive viewer (may need installation)	<code>htop</code>
pgrep <name>	Search for process by name	<code>pgrep nginx</code>
pidof <name>	Show PID of process	<code>pidof sshd</code>

ps aux OUTPUT EXPLAINED	
COLUMN	MEANING
USER	User who started the process
PID	Process ID
%CPU	CPU usage
%MEM	Memory usage
VSZ	Virtual memory size
RSS	Resident memory size
TTY	Terminal (or ? for none)
STAT	Process status
START	Start time
TIME	CPU time used
COMMAND	Command that started the process

PROCESS MANAGEMENT COMMANDS

COMMAND	DESCRIPTION	EXAMPLE
kill <PID>	Terminate a process (default SIGTERM)	kill 1234
kill -9 <PID>	Force kill a process (SIGKILL)	kill -9 1234
pkill <name>	Kill processes by name	pkill nginx
killall <name>	Kill all processes with the name	killall apache2
nice -n <num> <cmd>	Start a process with adjusted priority (lower value = higher priority)	nice -n 10 tar -czf backup.tgz /data
renice <num> -p <PID>	Change priority of a running process	renice -n 5 -p 1234

Use kill-9 only when necessary. It cannot be caught or ignored by the process.

JOB MANAGEMENT (SHELL JOBS)

COMMAND	DESCRIPTION	EXAMPLE
command &	Run command in background	sleep 100 &
jobs	List background and stopped jobs	jobs
fg %jobid	Bring job to foreground	fg %1
bg %jobid	Resume job in background	bg %1
Ctrl + Z	Suspend foreground job	(Press Ctrl + Z)
kill %jobid	Kill a job	kill %1
disown %jobid	Remove job from shell job table	disown %1

Job ID is usually shown as [number] or %number (e.g., %1, %2).

JOB CONTROL EXAMPLES

- Run a long task in background
`$ tar -czf backup.tgz /data &` # Job started in background
- Check running jobs
`$ jobs` # See job list
- Pause the job
`$ Ctrl + Z` # Job stopped
- Resume in background
`$ bg %1` # Job continues in background
- Bring to foreground
`$ fg %1` # Job is now in foreground

PROCESS STATES (STAT COLUMN)

STAT	MEANING
R	Running or runnable Actively running or ready to run
S	Sleeping (interruptible) Waiting for an event (e.g., I/O)
D	Uninterruptible sleep Waiting for I/O, cannot be interrupted
T	Stopped Stopped by job control signal
Z	Zombie Terminated but not reaped by parent
<	High priority Higher priority than normal
N	Low priority Lower priority than normal
+	In foreground process group Part of foreground job

EXAMPLE OUTPUT (ps aux)

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	11	0.0	0.1	165M	11M	?	Ss	10:00	0:02	systemd
root	876	0.0	0.2	348M	18M	?	Ssl	10:01	0:01	sshd
wale	1520	1.2	2.5	1650M	120M	pts/0	R+	10:10	0:05	python
wale	1890	0.0	0.1	218M	6.5M	pts/0	S	10:11	0:00	bash
wale	2345	0.0	0.0	0	0	?	Z	09:59	0:00	(nginx) (defunct)

STAT LEGEND

- Running
- Sleeping
- Uninterruptible Sleep
- Stopped
- Zombie
- High Priority
- Low Priority
- Foreground

USEFUL COMMANDS QUICK LIST

uptime	Show system uptime and load average
w	Show who is logged in and what they are doing
whoami	Show current user
time <cmd>	Measure how long a command takes
free -h	Show memory usage
df -h	Show disk usage
vmstat 1 5	System performance report (every 1 sec, 5 times)
iostat	CPU and I/O stats (sysstat package)

BEST PRACTICES

- Always check before killing a process.
- Use nice and renice to manage priority instead of killing.
- Run long tasks in background.
- Monitor system resources regularly.
- Avoid leaving zombie processes.
- Automate process management using scripts or tools.



QUICK TIP

Use htop for a better, user-friendly view of processes.

Install: `sudo apt install htop` (Debian/Ubuntu)

`sudo dnf install htop` (RHEL/CentOS/Fedora)



LINUX ENVIRONMENT VARIABLES



Environment variables store information that affects the behavior of processes and shells. They are inherited by child processes and are essential for configuration, paths, and system behavior.

KEY CONCEPTS



ENVIRONMENT VARIABLE

A dynamic value that affects the way processes and programs run.



INHERITANCE

Child processes inherit environment variables from their parent process.



SCOPE

Variables can be set for the current session, user account, or system-wide.



SECURITY

Be careful with sensitive values like passwords or API keys.



USE CASES

Configure paths, system behavior, applications, licenses, and more.

COMMON ENVIRONMENT VARIABLES

VARIABLE	DESCRIPTION	EXAMPLE VALUE
PATH	Directories to search for executable commands	/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
HOME	Home directory of the current user	/home/ubuntu
USER	Current username	ubuntu
SHELL	Default shell	/bin/bash
PWD	Current working directory	/home/ubuntu/projects
OLDPWD	Previous working directory	/home/ubuntu
LANG	Language and locale setting	en_US.UTF-8
TERM	Terminal type	xterm-256color
HOSTNAME	System hostname	devops-server
PATH	See all available variables	printenv

SET / EXPORT EXAMPLES

1. Set a variable for current session

```
MY_VAR="Hello World"
echo $MY_VAR
```

Output: Hello World

2. Export a variable (make it available to child processes)

```
export MY_VAR="Hello World"
child_command
```

Child process can access MY_VAR

3. Set and export in one line

```
export MY_VAR="Hello World"
```

4. Use a variable

```
echo "Home directory is $HOME"
```

Output: Home directory is /home/ubuntu (example)

VIEWING ENVIRONMENT VARIABLES

COMMAND	DESCRIPTION	EXAMPLE
printenv	Show all environment variables	printenv
env	Show environment variables	env
echo \$VAR	Show value of a variable	echo \$PATH
set	Show all shell variables and functions	set
declare -x	Show all exported variables	declare -x
declare -p VAR	Show details of a variable	declare -p PATH



Variable names are case-sensitive.

UNSET, MODIFY & PERSISTENT VARIABLES

1. Unset a variable

```
unset MY_VAR
```

Removes the variable

2. Modify a variable

```
MY_VAR="New Value"
```

Overwrites the value

3. Make a variable persistent

Add to your shell profile file (e.g., ~/.bashrc, ~/.zshrc)

```
# Example: Add to ~/.bashrc
export MY_VAR="Hello World"
```

Saved and loaded in new sessions

After editing profile files, run: `source ~/.bashrc` (or log out and log back in)

USEFUL EXAMPLES

Add directory to PATH (temporary)

```
export PATH="$PATH:/opt/myapp/bin"
```

Add to current session

Add directory to PATH (persistent)

```
echo 'export PATH="$PATH:/opt/myapp/bin"' >> ~/.bashrc
```

Persistent for new sessions

Check Java Home

```
echo $JAVA_HOME
```

Set Java Home

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
```

Check Python Path

```
echo $PYTHONPATH
```

Set Custom Variable

```
export APP_ENV=production
```

Example for apps

WHERE ENVIRONMENT VARIABLES ARE STORED / LOADED FROM



System-wide (ALL USERS)	/etc/environment Set by system. Loaded at boot.
System profile	/etc/profile, /etc/profile.d/*.sh Executed for login shells.
User profile (Login shell)	~/.bash_profile or ~/.profile or ~/.zprofile Executed for login shells.
User shell config (Interactive shell)	~/.bashrc (Bash) ~/.zshrc (Zsh) Executed for interactive shells.

Login shell reads profile files first, then interactive shell reads ~/.bashrc (if configured).

PRECEDENCE (ORDER OF PRIORITY)

- 1 Temporary variables in current session (export VAR=value)
- 2 Variables set in shell profile files (~/.bashrc, ~/.bash_profile, /etc/profile, etc.)
- 3 System-wide variables (/etc/environment)
- 4 Default values compiled into applications

BEST PRACTICES

- ✓ Use meaningful and clear variable names.
- ✓ Avoid overwriting system variables.
- ✓ Do not store sensitive information in environment variables (use secrets management).
- ✓ Document custom variables for teams.
- ✓ Use separate configs for dev, test, and prod.
- ✓ Keep persistent variables in profile files, not in scripts.



QUICK TIP

To see all environment variables in a nicely formatted way:

```
printenv | sort
```

Sorted list



SECURITY NOTE

Environment variables are visible to other users on the system in some cases (e.g., via `envu` or `/proc`). Do not store secrets like passwords or API keys.



LINUX INPUT, OUTPUT, PIPES AND REDIRECTION



Linux treats everything as a file. Commands read input (stdin), produce output (stdout) and errors (stderr). You can redirect these streams and connect commands using pipes.

KEY CONCEPTS



STDIN (0)

Standard input – data coming into a command.



STDOUT (1)

Standard output – normal output from a command.



STDERR (2)

Standard error – error messages from a command.



PIPE (|)

Pass the output of one command as input to another.



REDIRECTION

Change where input or output is read from or written to.



By default: stdin = keyboard, stdout = screen, stderr = screen

REDIRECTION OPERATORS

OPERATOR	MEANING	EXAMPLE	DESCRIPTION
>	Redirect output	ls > files.txt	Overwrite file with stdout
>>	Append output	ls >> files.txt	Append stdout to file
2>	Redirect error	cmd 2> error.log	Redirect stderr to file
2>>	Append error	cmd 2>> error.log	Append stderr to file
&>	Redirect all	cmd &> all.log	Redirect stdout and stderr
>>&	Append all	cmd >>& all.log	Append stdout and stderr
<	Redirect input	sort < names.txt	Use file as stdin
<<	Here document	cmd << EOF	Multi-line input to command
<<<	Here string	cmd <<< "text"	Single-line input to command
	Pipe	cmd1 cmd2	Pass stdout of cmd1 to stdin of cmd2
&	Pipe all	cmd1 & cmd2	Pipe stdout and stderr

FILE DESCRIPTORS

FD	STREAM	DEFAULT	DESCRIPTION
0	stdin	Keyboard	Input to the command
1	stdout	Screen	Normal output
2	stderr	Screen	Error output

STREAM COMBINATION EXAMPLES

```
cmd > out.txt 2>&1 # stdout and stderr to file
cmd 2>&1 > out.txt # stderr to stdout, then
                  # stdout to file (errors lost)
cmd &>> out.txt    # append both stdout and
                  # stderr to file
```

i Order matters when combining redirections.

INPUT REDIRECTION (<)

Redirect input of a command from a file or other source.

SYNTAX	EXAMPLE	DESCRIPTION
cmd < file	sort < names.txt	Use file as input
cmd << DELIM	cat << EOF Hello World EOF	Here document (multi-line)
cmd <<< string	grep foo <<< "foo bar baz"	Here string (single-line)

i Useful for feeding data into commands that normally read from keyboard.

OUTPUT REDIRECTION (>)

Redirect output of a command to a file.

SYNTAX	EXAMPLE	DESCRIPTION
cmd > file	ls > files.txt	Overwrite file
cmd >> file	ls >> files.txt	Append to file
cmd 2> file	ls 2> error.log	Redirect errors
cmd 2>> file	ls 2>> error.log	Append errors
cmd &> file	ls &> all.log	Redirect all output
cmd >>& file	ls >>& all.log	Append all output

i If file does not exist, it will be created.

PIPES (|) AND FILTERS

Pipes connect the output of one command to the input of another.

SYNTAX	EXAMPLE	DESCRIPTION
cmd1 cmd2	ps aux less	View long output page by page
cmd1 cmd2 cmd3	cat file grep error wc -l	Count lines containing 'error'
cmd 2>&1 cmd2	ls /notfound 2>&1 less	Pipe errors as well

Common Filters

grep	Filter lines	wc	Count (lines/words/chars)
sort	Sort lines	cut	Cut columns
uniq	Remove duplicates	awk	Pattern scanning & processing
		sed	Stream editor

PRACTICAL EXAMPLES

1 Save command output to a file

```
df -h > disk_usage.txt # overwrite
df -h >> disk_usage.txt # append
```

2 Save errors to a file

```
find /root -type f 2> find_error.log
```

3 Save output and errors to a file

```
apt update &> update.log
```

4 Send both output and errors to another command

```
ls /notfound 2>&1 | less
```

5 Count lines with word "error"

```
cat app.log | grep error | wc -l
```

6 Use file as input

```
sort < names.txt
```

7 Here document (multi-line input)

```
cat << EOF
Line 1
Line 2
Line 3
EOF
```

8 Here string (single-line input)

```
grep root <<< "root:x:0:0:root:/root:/bin/bash"
```

COMBINING PIPES & REDIRECTION

```
cmd1 | cmd2 > out.txt # pipe output to cmd2,
                      # then save stdout of cmd2

cmd1 | cmd2 2> err.log # save errors of cmd2

cmd1 2>&1 | cmd2 > out.txt # pipe all output of cmd1
                          # (including errors) to cmd2,
                          # then save

cmd1 |& cmd2 > all.txt # pipe stdout and stderr
                      # together (shorthand)
```

BEST PRACTICES

- ✓ Always quote variables in scripts.
- ✓ Use > to overwrite and >> to append.
- ✓ Log errors separately for troubleshooting.
- ✓ Use pipes to build powerful command chains.
- ✓ Test commands before putting in scripts.
- ✓ Document redirections in scripts.



QUICK TIP

Use **man bash** and search for "REDIRECTION" for complete details.



SEARCHING AND PROCESSING TEXT



Linux provides powerful commands to search for patterns and process text in files and streams.

KEY CONCEPTS



SEARCH

Find patterns (strings/regex) in files or input.



PROCESS

Transform, filter or manipulate text based on rules.



FILTER

Show only lines or fields that match conditions.



PIPELINE

Combine commands using pipes (|) to build powerful text-processing chains.



AUTOMATE

Use these tools in scripts to analyze logs, configs and data.



Most commands can read from files or standard input (stdin).

grep — SEARCH PATTERNS

Search for PATTERN in files or input.

SYNTAX	EXAMPLE	DESCRIPTION
grep pattern file	grep error app.log	Print lines containing 'error'
grep -i pattern file	grep -i error app.log	Case-insensitive search
grep -n pattern file	grep -n error app.log	Show line numbers
grep -v pattern file	grep -v debug app.log	Invert match (exclude)
grep -r pattern dir	grep -r TODO /etc	Recursive search in directory
grep -E 'regex' file	grep -E 'error warn' app.log	Extended regex
grep -w word file	grep -w user app.log	Match whole word
grep -c pattern file	grep -c error app.log	Count matching lines
grep -A N pattern file	grep -A 3 ERROR app.log	Show N lines after match
grep -B N pattern file	grep -B 2 ERROR app.log	Show N lines before match
grep -C N pattern file	grep -C 2 ERROR app.log	Show N lines before & after



Basit regex: . * ^ \$ [] [^] \

find — SEARCH FILES

Find files and directories by name, type, size, time, content and more.

SYNTAX	EXAMPLE	DESCRIPTION
find /path -name "*.log"	find /var/log -name "*.log"	Find files by name
find /path -type f	find /etc -type f	Find regular files
find /path -type d	find /home -type d	Find directories
find /path -mtime -7	find . -mtime -7	Modified in last 7 days
find /path -size +100M	find / -size +100M	Larger than 100 MB
find /path -name "*.conf" -exec grep -H "key" {} \;	find /etc -name "*.conf" -exec grep -H "listen" {} \;	Search text inside files
find /path -empty	find /tmp -type d -empty	Find empty directories
find /path -perm 644	find /etc -perm 644	Find files with permissions
find /path -user user1	find /home -user wale	Find files by owner



Use -exec command {} \; to run a command on results.

sed — STREAM EDITOR

Stream editor for filtering and transforming text.

SYNTAX	EXAMPLE	DESCRIPTION
sed 's/old/new/' file	sed 's/error/warning/' app.log	Replace first occurrence
sed 's/old/new/g' file	sed 's/error/warning/g' app.log	Replace all occurrences
sed -i 's/old/new/g' file	sed -i 's/foo/bar/g' file.txt	In-place replace
sed -n 'p' file	sed -n '1,5p' app.log	Print lines (with -n)
sed -n '/pattern/p' file	sed -n '/ERROR/p' app.log	Print lines matching
sed -n '/start/,/end/p' file	sed -n '/BEGIN/,/END/p' file	Print range of lines
sed '/^#/d' file	sed '/^#/d' config.conf	Delete lines starting with #
sed '/pattern/d' file	sed '/debug/d' app.log	Delete matching lines
sed '1d' file	sed '1d' file.txt	Delete first line
sed '\$d' file	sed '\$d' file.txt	Delete last line



Delimiters can be / or any character, e.g., sed 's|old|new|g' file

awk — PATTERN SCANNING & PROCESSING

Process text by pattern and fields.

SYNTAX	EXAMPLE	DESCRIPTION
awk '{print \$1}' file	awk '{print \$1}' users.txt	Print first column
awk -F: '{print \$1,\$3}' file	awk -F: '{print \$1,\$3}' /etc/passwd	Print fields 1 and 3 (delimiter :)
awk '/pattern/' file	awk '/ERROR/' app.log	Print lines matching pattern
awk '!/pattern/' file	awk '!/debug/' app.log	Print lines not matching
awk '\$3 > 100' file	awk '\$3 > 100' data.txt	Numeric condition
awk 'BEGIN{print "Hi"} {print}' file	awk 'BEGIN{OFS=","} {print \$1,\$2}' data.txt	BEGIN block / set output FS
awk 'sum+= \$3 END{print sum}' file	awk '{sum+=\$2} END{print sum}' sales.txt	Calculate sum of column
awk 'NR==1,NR<5' file	awk 'NR<5' app.log	First 5 lines
awk '{print \$NF}' file	awk '{print \$NF}' file.txt	Print last field



Useful variables: \$0 (whole line), \$n (nth field), NF (number of fields), NR (line no).

OTHER USEFUL TEXT COMMANDS

cut — Remove sections (columns)

SYNTAX	EXAMPLE	DESCRIPTION
cut -d: -f1 file	cut -d: -f1 /etc/passwd	Cut field 1 (delimiter :)
cut -d, -f1,3 file	cut -d, -f1,3 data.csv	Cut fields 1 and 3
cut -c1-10 file	cut -c1-10 file.txt	Cut characters 1 to 10

sort — Sort lines

SYNTAX	EXAMPLE	DESCRIPTION
sort file	sort names.txt	Sort alphabetically
sort -n file	sort -n numbers.txt	Sort numerically
sort -r file	sort -r file.txt	Sort reverse
sort -u file	sort -u file.txt	Sort and remove duplicates

uniq — Remove adjacent duplicates

SYNTAX	EXAMPLE	DESCRIPTION
uniq file	uniq file.txt	Remove adjacent duplicates
uniq -c file	uniq -c file.txt	Prefix lines by count
sort file uniq -c	sort file.txt uniq -c	Count all duplicates

PRACTICAL EXAMPLES (COMBINING COMMANDS)

1 Find ERROR lines in logs (with line numbers)

```
grep -n ERROR app.log
23: ERROR failed to connect
45: ERROR timeout occurred
78: ERROR invalid input
```

2 Count WARNING lines (case-insensitive)

```
grep -i WARNING app.log | wc -l
15
```

3 Show top 10 IPs by request count from access.log

```
awk '{print $1}' access.log |
sort | uniq -c | sort -nr |
head -n 10
1023 192.168.1.10
872 192.168.1.20
640 10.0.0.5
...
```

4 Replace old text in file (in-place)

```
sed -i 's/old_value/new_value/'
file.conf
# File updated
```

5 Find large files modified in last 7 days

```
find /var -type f -mtime -7
-size +100M -exec ls -lh {} \;
-rw-r--r-- 1 root root 25M May 20 4.log
-rw-r--r-- 1 root root 33M May 18 8.log
```

BEST PRACTICES

- ✓ Use quotes when patterns contain spaces or regex.
- ✓ Start with simple commands and build up.
- ✓ Use -n with sed to preview changes before editing.
- ✓ Combine commands with pipes for powerful results.
- ✓ Test on small data first, then scale up.
- ✓ Keep scripts readable and add comments.



QUICK TIP

Most commands read from stdin and write to stdout. Use pipes (|) to connect them.

```
cat file.txt | grep ERROR | sort | uniq -c
```

input filter sort count



REMEMBER

Every line is a string. Treat text as data!

